<usa> LstmEventFeatures (update_lstm_buffer)

«dataclass»
IsoForestResult

+ decision_function_raw: float
+ is_anomaly: bool
+ risk_score_iso: float

«module»
isoforest_service

- _to_accumulator(state: IsoDayState): DailyAggregateAccumulator
- _from_accumulator(acc: DailyAggregateAccumulator): IsoDayState
+ update_daily_aggregate(state: IsoDayState, event: dict, event_date: str, feature_set: str = "v1", tz: str | None = None) : tuple[IsoDayState, dict[str, float]]
+ feature_dict_desde(state: IsoDayState, feature_set: str = "v1") : dict[str, float]
+ score_isoforest(feature_dict: dict[str, float], artifacts: Artifacts): IsoForestResult

«module»
hybrid_service (orquestador)

+ SERVICE_ACCOUNT_VOLUME_THRESHOLD = 500_000
+ ACCION_SUGERIDA: dict[str, str]
+ ACCION_FUERA_DE_ALCANCE: str
+ ACCION_USUARIO_NUEVO = "alertar"
+ ACCION_SIN_MAPEO = "alertar"
+ W_PERSONAL_MAX = 0.7
+ EVENTOS_RECURSO_SENSIBLE = {file_created, file_written, file_renamed, file_deleted}
- _LATENCIA_SCORING_MS: deque[float] (maxlen=100)

+ process_event(uid: str, event: dict, artifacts: Artifacts): dict closures internas: _mutate(state), _mutate_iam(state)
- _registrar_latencia_scoring(ms: float): None
+ obtener_metricas_latencia_scoring(): dict
- _accion_para_nivel(nivel_sugerido: str): str
- _peso_personal(total_events: int, umbral_aprendizaje: int): float
+ w_iso_efectivo(coverage: str, artifacts: Artifacts): float
+ regla_piso_aplicada(risk_score_hibrido: float, nivel_sugerido: str, umbrales: dict): bool
+ personal_baseline_n(personal_baseline): int | None
+ construir_motivo(regla_piso_aplicada: bool, w_iso_efectivo: float, accion_atenuada: bool): str
+ accion_atenuada(en_aprendizaje: bool, tipo_evento: str | None): bool
- _fold_in_closed_day(anchor: HybridAnchorState, closed_day_date: str, closed_day_score: float, vida_media_dias: float): HybridAnchorState
- _derivar_detalle(event: dict): str | None
- _sin_score_response(uid: str, total_events: int, umbral_aprendizaje: int): dict

«module»
lstm_service

+ SEQUENCE_LENGTH = 50
+ LSTM_USER_PERCENTIL_PISO = 95.0

+ update_lstm_buffer(state: LstmState, event_features: LstmEventFeatures, artifacts: Artifacts) : tuple[LstmState, LstmChunkResult]
- _score_chunk(buffer: list[list[float]], artifacts: Artifacts): float
+ compute_lstm_user_risk(state: LstmState, artifacts: Artifacts): LstmUserRisk

«dataclass»
LstmChunkResult

+ completed: bool
+ reconstruction_error: float | None = None

«dataclass»
LstmUserRisk

+ lstm_risk_user: float | None
+ is_anomaly_lstm_user: bool

Dependencias en otras láminas
redis_state: UserScoringState,
load_state, update_state,
push_recent_event (lámina A)
iam_service, notifications (lámina C)
config_store, history_store (lámina D)

«module»
coverage

+ SERVICE_ACCOUNT_VOLUME_THRESHOLD = 500_000
+ COBERTURA_COMPLETA = "completa"
+ COBERTURA_PARCIAL = "parcial"
+ COBERTURA_SIN_SCORE = "sin_score"

+ determine_coverage(uid_type: str, total_events: int, lstm_chunks_completed: int): str

«dataclass»
LstmEventFeatures

+ type_encoded: int | None
+ hour_sin: float
+ hour_cos: float
+ dow_sin: float
+ dow_cos: float
+ time_delta_sec: float
+ country_changed: int
+ type_changed: int

+ as_vector(feature_set: str = "v1"): list[float]

«module»
features

+ FILE_EVENT_TYPES: set, LOGIN_EVENT_TYPES: set
+ ISOFOREST_FEATURE_COLS: list[str] (8, v1)
+ LSTM_SEQUENCE_FEATURES: list[str] (8, v1)
+ FILE_EVENT_TYPES_V2, SHARE_EVENT_TYPES_V2: frozenset
+ ISOFOREST_FEATURE_COLS_V2 (8), LSTM_SEQUENCE_FEATURES_V2 (7)
+ FEATURE_SETS: dict[str, tuple[list, list]] {"v1", "v2"}

+ feature_set_de(config: dict): str
+ columnas(feature_set: str): tuple[list[str], list[str]]
+ zona_horaria(): str env SCORING_ZONA_HORARIA
+ location_country(event: dict): str
+ parse_event_time(event: dict): datetime
+ compute_lstm_event_features(event_time: datetime, event_type: str, country: str, last_time: datetime | None, last_country: str | None, last_type: str | None, type_encoding: dict[str, int], feature_set: str = "v1") : LstmEventFeatures

«module»
personal_baseline_service

- _update_welford(stat: WelfordStat, x: float): WelfordStat
+ update_personal_baseline(state: PersonalBaselineState, iso_feature_dict: dict[str, float], lstm_feature_dict: dict | None, is_anomaly_iso: bool, is_anomaly_lstm_user: bool, iso_cols = ISOFOREST_FEATURE_COLS, lstm_cols = LSTM_SEQUENCE_FEATURES) : PersonalBaselineState (congelado si hay anomalía)
- _feature_z(stat: WelfordStat, x: float): float | None
- _rms_percentile(zs: list[float]): float
+ personal_risk_percentile(baseline: dict[str, WelfordStat], feature_cols: list[str], feature_dict: dict[str, float]): float | None
+ blend_con_personal(risk_poblacional: float, risk_personal: float | None, w_personal: float): float

«module»
drift_service

+ DRIFT_ROTATION_DAYS = 7
+ DRIFT_THRESHOLD_STD = 2.0

- _shift_std(anchor_stat: WelfordStat, current_mean: float): float | None
- _rotar(baseline: PersonalBaselineState, event_date: str) : PersonalBaselineState
+ check_and_rotate_anchor(baseline: PersonalBaselineState, event_date: str, iso_cols, lstm_cols) : tuple[PersonalBaselineState, list[DriftFinding]]

«dataclass»
DriftFinding

+ lado: str "iso" | "lstm"
+ feature: str
+ corrimiento_std: float

«dataclass»
DailyAggregateAccumulator

+ date: str
+ n_events: int = 0
+ distinct_types: set[str]
+ distinct_countries: set[str]
+ n_file_events: int = 0
+ n_login_events: int = 0
+ sum_hours: float = 0.0
+ n_off_hours: int = 0
+ type_counts: dict[str, int]
+ n_share_events: int = 0

+ add_event(event: dict, feature_set: str = "v1", tz: str | None = None): None
+ type_entropy(feature_set: str = "v1"): float
+ as_feature_dict(feature_set: str = "v1"): dict[str, float]

«module»
artifacts

+ SCRIPTS_DIR, FEATURE_DIR: Path
+ ISOFOREST_MODEL_DIR, SCORE_HIBRIDO_MODEL_DIR: Path
+ ARTIFACT_PATHS: dict[str, Path] (9 artefactos)
+ score_common, audit_common (de scripts/)
- _artifacts: Artifacts | None «singleton»

- _build_artifacts(): Artifacts
+ load_artifacts(): Artifacts (lifespan)
+ reload_artifacts(): Artifacts (hot-swap RF-21)
+ get_artifacts(): Artifacts

«dataclass»
Artifacts

+ isolation_forest: object isolation_forest.joblib
+ scaler_isoforest: object scaler_isoforest.pkl
+ lstm_autoencoder: object lstm_autoencoder.keras
+ lstm_get_target: Callable[[ndarray, ndarray], ndarray]
+ scaler_lstm: object scaler_lstm.pkl
+ type_encoding: dict[str, int] type_encoding.pkl
+ lstm_threshold: float lstm_threshold.pkl
+ config: dict config_score_hibrido.json (w_iso, vida_media_dias, umbrales, regla_piso)
+ iso_train_reference: ndarray
+ lstm_user_mean_error_train_reference: ndarray

«module»
explainability

+ TOP_N_FEATURES = 3
+ Z_SCORE_MINIMO = 1.0
+ DIRECCION_ENCIMA = "por_encima"
+ DIRECCION_DEBAJO = "por_debajo"

+ explicar_features(feature_dict: dict[str, float], artifacts: Artifacts): list[FeatureContribucion]

«dataclass»
FeatureContribucion

+ feature: str
+ z_score: float
+ direccion: str

<usa> determine_coverage()

<parámetro> artifacts de process_event, score_isoforest, update_lstm_buffer, explicar_features